

TensorHub: Scalable and Elastic Weight Transfer for LLM RL Training

Chenhao Ye[†], Huaizheng Zhang, Mingcong Han, Baoquan Zhong, Xiang Li, Qixiang Chen,
Xinyi Zhang, Weidong Zhang, Kaihua Jiang, Wang Zhang, He Sun,
Wencong Xiao, Andrea C. Arpaci-Dusseau[†], Remzi H. Arpaci-Dusseau[†]

ByteDance Seed

[†]University of Wisconsin–Madison

Abstract

Modern LLM reinforcement learning (RL) workloads require a highly efficient weight transfer system to scale training across heterogeneous computational resources. However, existing weight transfer approaches either fail to provide flexibility for dynamically scaling clusters or incur fundamental data movement overhead, resulting in poor performance.

We introduce Reference-Oriented Storage (ROS), a new storage abstraction for RL weight transfer that exploits the highly replicated model weights in place. ROS presents the illusion that certain versions of the model weights are stored and can be fetched on demand. Underneath, ROS does not physically store any copies of the weights; instead, it tracks the workers that hold these weights on GPUs for inference. Upon request, ROS directly uses them to serve reads. We build TensorHub, a production-quality system that extends the ROS idea with topology-optimized transfer, strong consistency, and fault tolerance. Evaluation shows that TensorHub fully saturates RDMA bandwidth and adapts to three distinct rollout workloads with minimal engineering effort. Specifically, TensorHub reduces total GPU stall time by up to 6.7× for standalone rollouts, accelerates weight update for elastic rollout by 4.8×, and cuts cross-datacenter rollout stall time by 19×. TensorHub has been deployed in production to support cutting-edge RL training.

1 Introduction

Reinforcement learning (RL) is central to building state-of-the-art large language models (LLMs) [5, 10, 14, 36]. With carefully designed rewards, RL boosts human alignment of LLM responses [6, 24], and improves reasoning [5, 10, 30], coding [19, 38], and mathematical capabilities [29, 32, 42].

Weight transfer is critical to LLM reinforcement learning. RL operates as a continuous feedback loop between training and inference: *trainer* workers produce new model weights at high frequency, and thousands of *rollout* workers pull new weights to run inference and generate responses for the next training step. In addition, upon failure recovery, a restarted worker must pull the latest weights before resuming tasks.

The increasing scale of production RL workloads requires a weight transfer system that can incorporate as many types of computational resources as possible, which is notably difficult today. For example, ByteDance production cluster

has abundant idle spot GPU instances, which can be preempted or restarted at any time; such a highly dynamic lifecycle makes them difficult to utilize. Multi-datacenter training is increasingly adopted by companies like Google and OpenAI [25, 44], but slow inter-datacenter links add more complexity.

Ideally, a weight transfer system should adapt to network topology, flexibly accommodate dynamic membership, and avoid unnecessary coordination and data movement, while remaining lightweight in resource usage. These properties, in sum, enable RL to harness heterogeneous resources and scale up for better intelligence.

However, existing weight transfer approaches fail to satisfy these goals. Collective communication (e.g., NCCL [23]) delivers high throughput, but its reliance on static communication groups makes it brittle under failures and ill-suited for dynamic clusters. Point-to-point communication (e.g., UCX [31]) offers flexibility, yet suffers from network contention under fan-out. Distributed storage systems (e.g., parameter servers [15, 17], Ray object store [21]) provide a clean decoupling between trainers and rollouts: trainers push new weights to storage, and rollouts pull on demand, all with minimal coordination; rollouts may scale up and down without trainers’ awareness. Yet, this push-then-pull pattern doubles data movement, which is prohibitively expensive for TB-scale LLM weights, along with significant storage/memory space overhead.

We seek to retain the decoupling advantages of storage interfaces while eliminating the overhead of extra data movement and storage space. This is made possible by an important observation: each version of the model weights is *immutable* and *highly replicated* for data-parallel inference. Such redundancy naturally provides multiple equivalent data sources to serve read requests for that version.

Our key idea is to utilize highly replicated model weights *in place* and provide access to them efficiently via a storage interface, called **Reference-Oriented Storage (ROS)**. ROS presents the illusion that certain versions of the model weights are stored and can be fetched on demand. Underneath, ROS does not physically store any copies of the weights; instead, it tracks the workers that hold these weights on GPUs for inference. Upon request, ROS directly uses them to serve reads. By avoiding explicit data ownership and extra

copies, ROS combines the flexibility and simplicity of storage with the efficiency of direct transfer.

Building such a system is challenging due to *integrity and availability requirements*. ROS addresses these challenges with two novel mechanisms: a *mutability contract* and a *retention protocol*. The mutability contract disciplines the mutation semantics that enable safe weight buffer reuse. The retention protocol defines which versions must be kept; in a corner case where a desired version may be lost, it offloads a copy to CPU memory to fulfill availability requirements.

We build **TensorHub**, which deploys the ROS approach in a production system with *scalable data transfer, strong consistency for model parallelism, and fault tolerance*. It schedules transfers with topology awareness and leverages pipelined replication to scale throughput. TensorHub enforces a consistent view among workers within each model-parallel group to ensure correctness. TensorHub proactively detects failures and orchestrates recovery, enabling transparent operation under dynamic cluster scaling.

We evaluate TensorHub on large-scale RL workloads spanning dozens to a thousand GPUs, elastic clusters with spot churn, and multi-datacenter topologies. In a 1024-GPU training task, TensorHub reduces total GPU stall time by up to 6.7× compared to NCCL. With an elastic cluster, TensorHub’s dynamic load balancing yields 4.8× faster updates compared to UCX. In a cross-datacenter setting, TensorHub cuts GPU stall time by 19× by avoiding redundant TCP transfers.

TensorHub has been deployed in ByteDance production to support cutting-edge RL training. With its strong performance and extensive functionality, TensorHub demonstrates a new way to build RL systems that operate effectively and efficiently at scale.

In summary, this paper makes the following contributions:

- Reference-Oriented Storage (ROS): an ownership-free storage architecture specialized for model weights.
- TensorHub: a production-quality system that extends ROS with scalable transfer, strong consistency, and fault tolerance.
- Three case studies on diverse rollout workloads to demonstrate TensorHub’s performance and functionality benefits over other production baselines.

2 Background and Motivation

In modern LLM reinforcement learning (RL) workloads, weight transfer is essential for fully leveraging heterogeneous compute resources. This section covers the background of RL workloads, the requirements of a weight transfer system, and the limitations of existing approaches.

2.1 Reinforcement Learning Background

RL employs a more complex iterative process than pretraining, centered on two modules: *rollouts* and the *trainers*. A typical step of RL algorithms [28, 32] proceeds as follows: ❶ Generation: The rollout workers consume a batch of prompts and

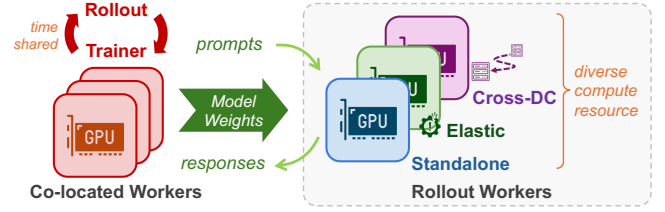


Figure 1. RL Workload with Diverse Rollout Strategies. Rollout workers stream in prompts and model weights and stream out responses. While prompts and responses are light text, the model weights can be several terabytes in size, necessitating an efficient transfer system.

produce responses through inference. ❷ Scoring: Generated responses are evaluated by a reward function (e.g., human ratings [1, 4, 24], model-based scoring [4, 24], or rule-based rewards [32]). ❸ Training: The trainer workers compute the gradients and update the model weights. ❹ Weight transfer: The newly trained weights are resharded and converted to inference-ready format, then transferred from trainers to rollouts for the next step. This workflow creates continuous, frequent demands for weight transfer across large clusters. All subsequent weight-transfer patterns stem directly from this iterative structure.

RL Workloads in Production. Production RL frameworks require weight transfer to incorporate as many types of rollout strategies and computational resources as possible.

In a basic *co-located* architecture, the rollouts and the trainers are physically placed on and time-share the same set of GPUs. This design is adopted by veRL [34], ReaL [20], and RLHFuse [45] for its simplicity and high resource utilization. However, the rollout time can account for more than 90% of the training time [11, 12, 34], thereby becoming the bottleneck. This motivates a series of strategies to scale out rollout throughput with more resources (as shown in Figure 1):

- *Standalone rollouts* are a set of GPU workers dedicated to running rollout tasks. This design, adopted by many frameworks [7, 12, 33, 39], substantially speeds up rollout.
- *Elastic rollouts* run on spot GPU instances, which may be preempted or restarted at any time. With a lower price, they offer better cost efficiency. RLBoost [40] reports that this approach can improve training throughput by up to 1.97×.
- *Cross-datacenter rollouts* are designed to support hyper-scale training [25] and exploit heterogeneous resource availability [44]. Because GPUs are often procured and deployed in different batches, inference-optimized GPUs may not be available in the same datacenter as the trainers [44]. Using those resources requires cross-datacenter transfer.

These diverse rollout workers can be viewed as a pipeline that streams in prompts and model weights and streams out responses. The prompts and responses are lightweight text, while the model weights can reach terabyte scale, making an efficient transfer system essential.

Approaches	Example	Topology Awareness	Elasticity	Minimal Coordination	Minimum Data Movement	No Extra Space
Collective Comm.	NCCL [23]	✓ (only if single DC)			✓	✓
P2P Comm.	UCX [31]		✓		✓	✓
Distributed Storage	PS [17], Ray object store [21]	✓	✓	✓		
	TensorHub	✓	✓	✓	✓	✓

Table 1. Comparison of Different Weight Transfer Approaches.

2.2 Weight Transfer Requirements

To serve these diverse rollout strategies, an ideal weight transfer system must satisfy five goals:

Goal 1: Topology Awareness. To fully utilize network bandwidth, the weight transfer system must incorporate network topology and runtime load knowledge. For cross-datacenter training, it must minimize the network traffic over the slow inter-datacenter links. Within the same datacenter, it should avoid congestion on a single node’s bandwidth.

Goal 2: Elasticity. The weight transfer system must support a dynamic cluster that scales up and down while transparently masking failures. If a worker fails or is preempted, the system must safely evict it without disrupting healthy workers. If a worker restarts or joins the cluster, the system must promptly serve weights on demand.

Goal 3: Minimal Coordination. Weight transfer should minimize coordination among workers. Coordination not only complicates programming but also introduces additional waiting and amplifies stragglers at scale.

Goal 4: Minimum Data Movement. The optimal data transfer must occur directly between the source and destination GPUs, with no intermediate hop. An extra hop implies additional network/PCIe costs, hurting overall efficiency.

Goal 5: No Extra Storage/Memory Space. As LLM weights can be several terabytes, storing extra copies requires massive storage or memory space. Such extra resource requirements should be as light as possible for cost-effectiveness.

2.3 Limitations of Existing Approaches

Existing weight transfer approaches fall under three main categories: *collective communication*, *point-to-point communication*, and *distributed storage*. Table 1 summarizes how well each meets the above goals.

Collective Communication. NVIDIA Collective Communication Library (NCCL) [23] is the most widely used approach for weight transfer, adopted by veRL [34], OpenRLHF [12], AReaL [7], vLLM [16], SGLang [43], Slime [48], among others. In these frameworks, trainers broadcast weights to rollouts via NCCL collectives, achieving high throughput with topology-aware transfer that saturates RDMA bandwidth.

However, collective communication fails on the elasticity and minimal coordination goals. First, collective communication requires a pre-defined communication group that cannot add/remove nodes afterward. This prevents it from

being applied to a dynamic cluster where nodes may fail, be preempted, or be restarted; any failure will cause the entire cluster to crash. Second, broadcast requires all nodes to cooperate, which in practice translates to a global barrier in RL frameworks that interrupts all workers and does not resume execution until all transfers have finished [7, 12, 34]. This amplifies stragglers, which are common in RL workloads. For example, a rollout may delay responding to an interrupt while generating a long sequence, or network contention may slow a subset of nodes during transfer.

Point-to-Point Communication. Point-to-point (P2P) communication (e.g., UCX [31]) enables flexible data transfer between workers without a communication group, and thus naturally supports dynamic clusters. In an ideal case, such direct transfer can saturate the RDMA bandwidth between two endpoints, delivering high throughput.

However, P2P communication is less commonly used in RL because it has no global view of the runtime topology and therefore cannot balance load effectively. Senders serve requests independently, making their outbound bandwidth the bottleneck under fan-out. For example, when multiple rollouts pull weights from a trainer, they contend for the trainer’s uplink bandwidth, degrading performance. Moreover, P2P still requires coordination at the framework level. When a trainer issues a `send()`, the corresponding rollout must be interrupted to invoke `recv()`; the `send()` blocks until the receiver cooperates.

Distributed Storage. Distributed storage systems, e.g., parameter servers (PS) [15, 17] and Ray Plasma object store [21], provide a compelling *decoupled* interface for weight transfer: trainers *push* updated weights to storage, and rollouts *pull* weights on demand. By replicating weights across storage nodes, such systems can exploit topology and locality, serving requests from nearby or less-loaded replicas. Moreover, this decoupling also naturally supports elasticity, since rollouts can join, leave, or restart independently without trainers’ awareness. Finally, trainers can proceed immediately after pushing weights, while rollouts fetch weights on demand, minimizing coordination between them.

Despite these compelling properties, storage has not been adopted as the primary weight transfer mechanism in major LLM RL frameworks due to poor performance. For parameter servers [15, 17], the push-then-pull workflow amplifies network traffic, and the storage server’s network bandwidth becomes a potential bottleneck at scale. Ray Plasma

object store [21] mitigates the network bottleneck by co-locating storage with workers, but still incurs extra data movement, requiring GPU–CPU copies and (de)serialization before transferring and reconstructing weights. For example, transferring 40 GB of data between two 8-GPU machines via the Ray object store takes 32 seconds in our measurement, whereas GPU-direct RDMA completes the same transfer in ~ 0.2 seconds. Such orders of magnitude performance loss cannot be justified by other functionality benefits. Lastly, storing TB-sized LLM weights is too expensive regardless for remote storage or co-located storage. In our experiment, the Ray object store commonly crashes due to out-of-memory when transferring >300 GB of data.

More fundamentally, data ownership is the root cause of storage’s extra movement and space overhead. Each write operation copies weights into storage-managed space (either a remote server or co-located CPU memory), allowing the system to manage data independently of workers. This ownership enables trainer-rollout decoupling but also introduces additional serialization, copying, network traffic, and storage overhead that ultimately degrade performance.

Summary. Existing approaches present a trade-off. Collective communication achieves high throughput with topology-aware transfer, but requires rigid membership and global coordination. P2P communication supports dynamic clusters and direct transfer, but struggles with load balancing and still incurs coordination at runtime. Distributed storage offers clean decoupling and flexibility, but pays the price of additional data movement and storage overhead due to full data ownership. As a result, none of these approaches simultaneously satisfies all five goals—topology awareness, elasticity, minimal coordination, minimum data movement, and no extra storage space.

3 Reference-Oriented Storage

Distributed storage offers a clean and flexible interface for weight transfer, but its reliance on data ownership introduces fundamental inefficiencies. To address this limitation, we present *Reference-Oriented Storage* (ROS), a new storage abstraction that does not own data.

ROS is enabled by three important properties of model weights in RL. First, weights are *highly replicated* due to data parallelism. This redundancy means there are multiple symmetric sources available for transfer. Second, weights used in inference are *immutable*. As a result, reading them from a remote worker’s GPU memory is safe and requires minimal coordination. Third, weight versions are *short-lived*. Training constantly generates new versions, and only the most recent few are relevant, so long-term durability is unnecessary.

ROS directly repurposes those model weights *in place* to provide the illusion that certain versions of the weights are stored and can be fetched on demand. Under the hood, ROS does not physically store any copies of the weights.

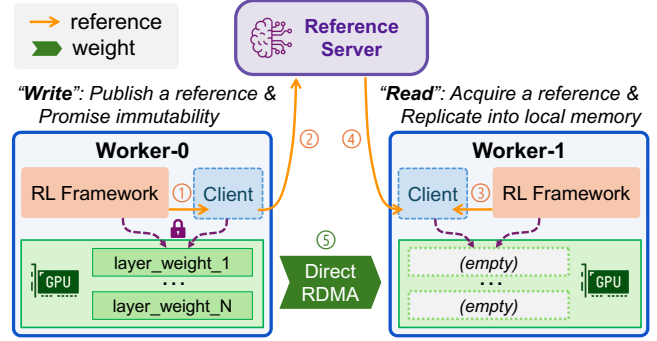


Figure 2. Reference-Oriented Storage Workflow. The server only operates on lightweight references. The bulk weight transfer is directly between clients.

Instead, it tracks the workers with these weights on GPUs for inference; upon request, ROS directly uses them to serve reads. By avoiding explicit data ownership and extra copies, ROS combines the flexibility of storage with the efficiency of direct transfer.

This section covers the elemental primitives required for a minimal ROS system. We first illustrate the overview of ROS and then elaborate on two key mechanisms required for integrity and availability. A more sophisticated system with rich features will be covered in §4.

3.1 ROS Overview

Figure 2 shows the architecture of ROS and the execution flows. ROS consists of a centralized reference server and a client library imported by each worker.

ROS exposes publish and replicate primitives for write and read operations. To write weights into ROS, Worker-0 calls publish and passes a reference to its in-GPU weights; its ROS client forwards the reference to the reference server. To read weights, Worker-1 invokes replicate with a reference to its local weight buffers. Its ROS client queries the server and acquires a source reference that points to Worker-0’s GPU memory; the client then issues RDMA to directly read weights from Worker-0’s memory into its local buffers. Upon completion, Worker-1’s copy also becomes another replica of the model weights. Future replication requests can also be served from either Worker-0 or Worker-1.

Critically, these data flows require no additional data movement or storage overhead. Unlike traditional distributed storage, the ROS server never stores or transfers weight data. It only operates on the lightweight references. The bulk data transfer is directly between the source and destination memory without intermediate hops or staging space.

ROS treats all copies of the same weight as symmetric replicas, whether created via publish or replicate. This means a rollout need not fetch weights from a trainer; it can retrieve them from any peer. This design enables an efficient and resilient peer-to-peer data dissemination. In the presence of failures, as long as at least one replica remains,

a restarted worker can recover from the live peer, effectively making the cluster self-healing.

Challenges. Since ROS only *references* weights but does not *own* them, it must address two challenges to be functional.

- *Integrity:* Weights require the flexibility for occasional mutations (e.g., reused for a new version of weights). ROS must define a clear mutation semantics to avoid data corruption.
 - *Availability:* Because ROS does not own the data, a weight version disappears if no worker holds a copy. The system must define which weights should remain accessible and introduce protocols to preserve that.
- ROS introduces two key mechanisms to address the challenges: the *mutability contract* and the *retention protocol*.

3.2 Integrity via Mutability Contract

ROS relies on the weight immutability for safe sharing, but RL training requires the flexibility for mutability. For example, as weights are too large to be duplicated on GPUs, a rollout may want to reuse the weight buffers in place when pulling the new weights. Similarly, a trainer must be able to mutate the weights to produce a new version.

ROS addresses this demand with the `unpublish` primitive and the *mutability contract*. When holding a replica, a worker commits to not mutate the weight data. When mutation is required, the worker must call `unpublish` to explicitly revoke the immutability commitment. Upon request, the reference server ensures this worker’s weights are no longer visible to others; if there is an ongoing transfer from this worker, the server waits to drain the transfer and only acknowledges success after the transfer completes. This ensures every concurrent transfer still receives uncorrupted data.

Fortunately, adhering to the mutability contract is easy to implement in practice. An RL worker’s execution flow is typically organized into stages, where the immutable boundary is straightforward to identify. In addition, Python is a memory-safe language, naturally avoiding many dangling-pointer corruptions.

3.3 Availability via Retention Protocol

ROS stores no copies, so if all workers unpublish a version, that version is unavailable. Such behavior is appropriate for overly stale weights, as they are not needed by any workers and thus safe to drop. However, losing the latest (few) versions of the weights is unacceptable. This scenario can occur in a corner case if all trainers unpublish the latest version before it is replicated by any rollout. To avoid such unavailability, the system requires clear semantics for “what data must be available” and a protocol to enforce that.

ROS fulfills this requirement with the *retention protocol*. ROS exposes a `retain` semantics where a worker can declare the versions it wants to keep available (e.g., the latest k versions). When processing `unpublish` requests, the server checks if the requesting replica is the last copy of a retained

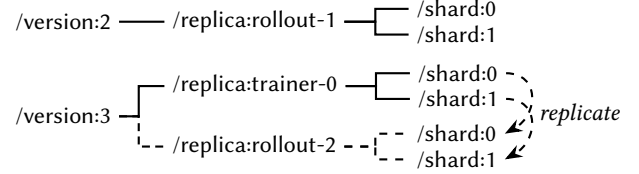


Figure 3. TensorHub Naming Scheme. Rollout-2 fetches version 3 by replicating a copy; after doing so, it also becomes a replica capable of serving subsequent replication requests.

version; if so, it notifies the client to offload a copy of weights to CPU memory and publish that as an *offload* replica.

Note offloading is a safeguard for corner cases and is rarely triggered in practice. Rollouts typically see and replicate the new version promptly; as long as one copy is replicated, trainers can unpublish without triggering any offload. When offloading does occur, only a single copy across the entire cluster is kept, minimizing memory overhead. Measurements show that offloading over the PCIe bus is very fast (~ 48 GB/s in our setup), resulting in a minor delay for unpublish (< 2 s even when offloading the entire GPU). The offload replica is automatically released once it has been replicated by another worker (and is thus no longer the last copy) or once a newer version’s publish makes it no longer retained.

The elegance of the retention protocol is that it provides similar availability as traditional ownership-based storage, while eliminating the ownership overhead in common cases.

3.4 ROS Summary

ROS achieves Goals 3, 4, and 5 by design. By providing a storage interface, ROS enables rollouts to pull weights on demand without trainers’ explicit coordination (Goal 3). By not taking ownership of weight data, ROS avoids the overhead of data movement (Goal 4) and extra storage space (Goal 5) in the common case.

ROS also establishes a good foundation for Goals 1 and 2. In principle, its centralized server architecture offers a global view of network topology and runtime load, making topology-aware transfer easier to implement (Goal 1). Elasticity can be achieved as long as the ROS system itself is fault-tolerant (Goal 2). In summary, both goals fit into the ROS’s design but require a well-optimized implementation.

4 TensorHub Design and Implementation

We build **TensorHub**, a production-quality system that extends the ROS idea with rich features required by real-world use cases. As LLM model sizes increase, model parallelism is required, with each GPU worker holding only one shard of weights. Therefore, sharding must be natively incorporated in addition to the previous ROS requirements.

Overall, a production-quality system must satisfy three practical requirements. First, for efficient transfer at scale, the system must schedule transfer based on the network

Name	Description
<code>open(...)</code> $\rightarrow$ <code>ShardHandle</code>	Acquire a shard handle. Optionally, declare the desired versions to retain for the retention protocol.
<code>register(named_tensors)</code>	Register weight tensors.
<code>unregister()</code>	Unregister weight tensors.
<code>publish(version)</code>	Publish a reference to the registered tensors under the given version.
<code>unpublish()</code>	Revoke the previously published reference.
<code>replicate(version)</code>	Replicate the given version of weights into registered tensors. <i>Support relative versions.</i>
<code>update(version)</code> $\rightarrow$ <code>bool</code>	Atomically check whether the given version is available and differs from the current one; if so, unpublish the current version and replicate the new one; return whether an update occurred. <i>Support relative versions.</i>
<code>list()</code> $\rightarrow$ <code>Dict</code>	Returns a dictionary mapping each available version to its set of replica names.
<code>wait(predicate)</code>	Wait until <code>predicate(list())</code> is evaluated to true.
<code>close()</code>	Destroy the handle; automatically unpublish and unregister if applicable.

Table 2. TensorHub APIs. `open()` returns a handle, and the rest of the APIs are methods of this handle.

topology and runtime loads. Second, for correctness, the system must guarantee that workers in the same model-parallel group see a consistent view of the system state, preventing control flow divergence. Third, to support dynamic cluster scaling up and down, it must gracefully handle failures.

This section begins with the naming (§4.1) and APIs (§4.2) and then introduces topology-aware transfer (§4.3), model-parallel consistency (§4.4), and fault tolerance (§4.5).

4.1 Naming Scheme

We begin with a hierarchical naming scheme to establish a precise vocabulary. In TensorHub, each model forms an independent domain, managed by a single reference server. Each model evolves over *versions*, each produced by one step of training. A version consists of one or more *replicas*, each representing a full copy owned by a model-parallel process group. Finally, each replica is divided into *shards*, with each shard owned by a single worker. Figure 3 illustrates an example with three replicas, each composed of two shards.

Each version is uniquely identified by an integer, called *absolute* version. TensorHub also supports *relative* versions, where strings “latest” or “latest- k ” are resolved to the latest version or the latest minus k , respectively. Relative versions are useful because RL typically only cares about freshness relative to the latest one. Off-by- k -step RL training workloads [7, 33, 39, 41, 47] require multiple versions to co-exist in the system, where the “latest- k ” is particularly useful.

4.2 TensorHub APIs

TensorHub APIs are summarized in Table 2. As the same sets of tensor buffers are commonly used across versions, TensorHub exposes `ShardHandle` to operate them.

- **Open.** A worker begins by calling `tensorhub.open()` to acquire a handle for a particular shard within a replica. If it wishes to retain certain versions, it specifies so upon `open`.
- **Registration.** Before publishing or replicating, a worker must register weight tensors with the handle; if it later deallocates these tensors, it must call `unregister()`. These APIs uniformly support both CPU and GPU tensors, which offers

<pre>import tensorhub handle = tensorhub.open(model_name="actor", replica_name="trainer-0", num_shards=WORLD_SIZE, shard_idx=RANK,) tensors = get_weights_tensors() handle.register(tensors) for step in range(NUM_STEPS): handle.publish(version=step) do_rollout() handle.unpublish() do_train() # convert to the # inference-ready format post_process_weights() handle.close()</pre>	<pre>import tensorhub handle = tensorhub.open(model_name="actor", replica_name="rollout-1", num_shards=WORLD_SIZE, shard_idx=RANK, retain="latest",) tensors = get_weights_tensors() handle.register(tensors) # fetch the initial weights handle.replicate("latest") while True: is_updated = \ handle.update("latest") if is_updated: clear_kv_cache() do_rollout() handle.close()</pre>
(a) Co-located trainer & rollout.	(b) Standalone rollout.

Figure 4. TensorHub Examples.

flexibility in worker memory management. For example, in a fully disaggregated architecture where trainers execute training in a tight loop, they can publish weights on CPU.

- **Publish.** To make its registered tensors visible under version v , the worker calls `publish(v)` with an immutability commitment. When the worker desires mutation, it must invoke `unpublish()` first. These two calls form the core mutability discipline for reference safety.
- **Replicate.** A worker obtains weights by naming the desired version (absolute or relative). `replicate(v)` materializes version v into the handle’s registered tensors, blocking if necessary until the version exists.
- **Update.** With a version held, a worker can call `update(v)` to atomically transition to a newer version if available. Atomicity avoids races between checking the version and replicating it, enabling simple polling loops like `update("latest")` during rollout inference. This API is useful for a rollout to poll whether a new version of the weights is available; if not, it continues inference with the current weights.
- **Query.** TensorHub provides lightweight introspection through `list()` and `wait(predicate)`, which expose global

metadata (such as available versions or replicas) for implementing custom policies. For example, rollouts can discard excessively stale responses, or trainers can wait for lagging rollouts to catch up [7, 33, 39, 41, 47].

- **Close.** When a worker finishes all operations on the handle, it calls `close()` to release resources and clean up states.

Example. Code snippets in Figure 4 illustrate how TensorHub simplifies weight transfer in RL training. Figure 4a shows the workflow of a co-located worker that runs both training and rollout tasks. In each step, it publishes its weights before starting rollout; after completing a configured amount of work, it switches to training. Before updating the weights, it unpublishes them. After training completes, the worker begins the next step and publishes the updated weights.

Figure 4b shows an example of a standalone rollout. Upon initialization, it calls `replicate()` to fetch a copy of weights before starting inference. Then, periodically (e.g., after finishing a small batch of inference tasks), it polls TensorHub to check for and fetch a new version of the weights if available; if not, it continues inference with the existing weights.

4.3 Topology-Aware Replication at Scale

In RL training, every newly published version often needs to be fetched by multiple rollouts. A system that funnels all replication through a single node will collapse under network contention. TensorHub is therefore architected with a centralized reference server to schedule transfer based on network topology and runtime loads, while the clients remain decentralized for scalable data movement.

4.3.1 Load-balanced Scheduling at the Server. Efficient transfer begins with choosing the right source for each request. TensorHub centralizes scheduling on the server, exploiting its global view of the system for informed decisions.

The server always prioritizes a source replica located in the same datacenter. Within each datacenter, the server maintains a reference count for each replica—representing the number of replication requests it is currently serving—and always selects the least-loaded replica. Under pipeline replication (§4.3.3), each replica serves at most one request at a time, barring failures. This load-balancing policy also keeps unpublish latency low: if a source replica requests to unpublish, it needs to wait for in-flight replication to drain, which is brief and bounded by a single request. If no source replica is available within the same datacenter, the server falls back to cross-datacenter replication (§4.3.4).

In practice, a single server is sufficient to scale a training task to thousands of workers (more measurements in §5.2). As future work, if the server becomes a scalability bottleneck at even larger scales, TensorHub could partition workers into sub-clusters, each managed independently by a server. These servers would only need to lazily synchronize on critical events (e.g., a new version is published).

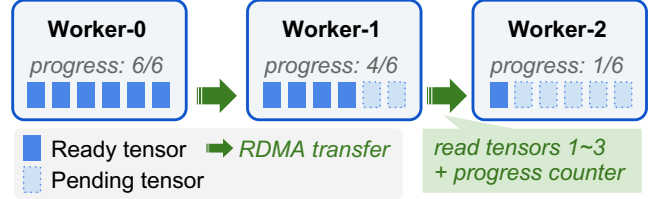


Figure 5. Pipeline Replication. Worker-0 is the only source, while both Worker-1 and Worker-2 are requesting data. To scale throughput, TensorHub schedules a pipeline where Worker-2 reads partially replicated data on Worker-1.

4.3.2 High-Performance Transfer between Clients.

TensorHub data-plane efficiency relies on a high-performance transfer method between clients. To achieve this, each client embeds a hardware affinity-aware transfer engine that uniformly supports RDMA and TCP transfers over both GPU and CPU memory. More specifically, the transfer engine offers three transfer modes, customized for workloads and available transport methods:

- **RDMA Direct:** In workloads where tensors remain allocated for long periods, the transfer engine registers tensor buffers directly with the RNIC. Remote clients can then issue one-sided RDMA reads to pull data directly. This zero-copy path bypasses CPUs entirely and achieves the best performance.
- **RDMA Copy:** When users frequently reallocate tensors, repeated RNIC registration is too costly. In this case, the transfer engine employs background threads to serve remote requests by copying the tensor data slice-by-slice into pre-registered buffers and then issuing RDMA writes. This approach eliminates re-registration overhead while still preserving RDMA’s high throughput.
- **TCP:** In environments where RDMA is unavailable (e.g., across datacenters), the transfer engine uses TCP.

The transfer engine is affinity-aware and optimized for modern multi-GPU, multi-NIC servers. For GPU-resident tensors, the engine selects the NIC closest to the tensor’s GPU; in RDMA copy mode, it uses the pre-registered RDMA regions on the same GPU devices.

Tiny-Tensor Optimization. LLM models contain a large number of tiny tensors, which are inefficient to register and transfer. To address this, TensorHub compacts all tiny tensors (<2 MB) into contiguous buffers; only these compacted buffers are registered and transferred. During replication, the receiver unpacks them into their corresponding slices.

This optimization reduces registration costs and improves bandwidth utilization, at the cost of very minor memory overhead. For example, a 36B model has approximately five hundred tensors, half of which are tiny. When compacted, they require only about 3 MB of extra memory in total, which is negligible for a 19 GB shard.

4.3.3 Pipeline Replication. Pipeline replication enables TensorHub to scale throughput beyond what a single source could provide. A key challenge in RL training is that large

bursts of rollouts may simultaneously request the latest version, creating extreme fan-out. Pipeline replication allows partially-replicated workers to begin serving others immediately, turning replication into an expanding pipeline rather than a single-rooted broadcast (as shown in Figure 5). Pipeline replication builds from the observation that most RDMA NICs are full-duplex, and therefore, when a NIC is receiving data, its uplink is idle and can serve data.

Pipeline replication uses the metadata and transfer mechanisms described below. Each replicating worker tracks a *progress counter* that indicates how many tensors of the target version it has received locally. When a new worker requests a version, the reference server selects the least-loaded source, which can be either fully replicated or still in progress. If the source is only partially replicated, the requester reads the source’s progress counter and then fetches the prefix that is available. Once completed, it re-reads the progress counter and repeats the process until all data is received.

Pipeline replication transforms the topology from a fixed N-way fan-out into a bandwidth-amplifying DAG, where each replicating worker increases the system’s bandwidth. Because transfers are decentralized and occur directly between peers, throughput grows with the number of active clients, and latency remains low even under heavy demand.

4.3.4 Cross-Datacenter Replication. TensorHub’s replication model generalizes to clusters spanning multiple datacenters, with a two-stage replication pattern.

When a new version is published, the first replica located in a different datacenter must fetch the data over TCP; we refer to this replica as the *seeding replica*. Once seeding completes, subsequent replicas in that datacenter can fetch the data over RDMA, enabling full pipeline replication locally without repeatedly invoking cross-datacenter transfer.

TCP-based seeding has substantially lower bandwidth than RDMA and can therefore stall pipeline replication if other replicas immediately try to update and fetch from the partially-filled seeding replica. To avoid this stall, TensorHub augments the `update()` API with *smart skipping*: if a polling replica observes that a seeding replica is actively fetching over TCP, it treats the new version as temporarily unavailable and retries later, allowing seeding to complete before datacenter-local replication begins. This avoids serializing the entire pipeline behind slow cross-datacenter transfer.

To further reduce GPU stall time for seeding replicas, TensorHub offers an *offload seeding* option. When seeding is required, the client library creates a temporary offload buffer in CPU memory and initiates cross-datacenter transfer into this buffer in the background. The `update()` call returns immediately, and subsequent polling prioritizes consuming the offloaded copy if completed. This approach trades a bounded amount of CPU memory for reduced GPU stall time by creating at most one offload seeding replica per datacenter.

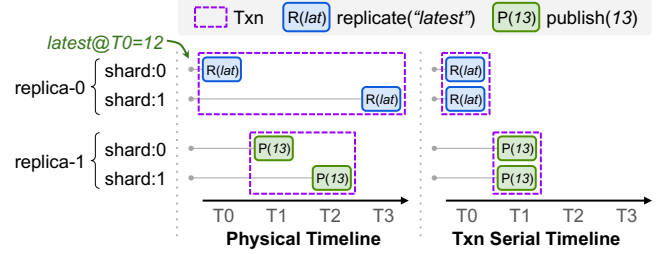


Figure 6. Sharding Consistency Example. At T_0 , shard:0 of replica-0 requests to replicate the latest and observes version 12. At T_1 and T_2 , replica-1 publishes version 13. Consequently, when shard:1 of replica-0 makes the same request at T_3 , it observes version 13 as the latest. Without coordination, this leads to divergence. TensorHub’s transactional semantics ensure replica-0’s requests both see version 12.

4.4 Consistency for Model-Parallel Sharding

LLM weights often exceed the memory of a single GPU, so model parallelism is required. In this setup, weights are sharded across a group of workers that must execute identical code, following an SPMD (single-program, multiple-data) paradigm. Divergence in a group risks hanging or corruption.

For correctness, all workers in a model-parallel group should ideally observe a consistent view of system states, such as whether version N is available or which version is the latest. In practice, however, requests from different workers may reach the server in an interleaved order and observe inconsistent states. Figure 6 shows one such example.

To address this issue, TensorHub server enforces transactional semantics at the granularity of each model-parallel group (i.e., replica). Specifically, when the first worker’s request arrives at the server, it starts a transaction, executes the requests on behalf of the entire group, and saves necessary information (e.g., the latest version it sees) into a transaction-local buffer. Subsequent requests consume the saved information for consistency. Effectively, transactions are serialized in the order they started.

4.5 Robustness to Churn and Failures

Large-scale training runs on clusters with highly variable reliability. Rollout workers on spot instances can be preempted at any time, and even non-spot nodes occasionally fail at scale. Our storage abstraction must therefore assume frequent replica churn and provide correctness without manual recovery. TensorHub is designed for this failure model: it detects failed replicas, safely re-routes ongoing replication, and integrates cleanly with our retention semantics.

General Failure Handling. TensorHub considers a client failed if no heartbeat arrives within the timeout window. Failure is handled at the replica granularity: the server removes the entire failed replica and releases associated resources.

Failures can occur during active replication. If a replica fails while fetching data, the timeout causes the server to invalidate the incomplete replica and release the reference

counter it acquired. Conversely, if a replica fails while acting as a source, the receiving replica detects the transfer failure and reports to the server. The server marks the source as failed, preventing future replication from it, and selects an alternate source to recover the transfer.

Retention under Frequent Churn. Spot instances change the failure model: failures are not rare events but the norm. To ensure retention with spot instances, when counting the replicas that satisfy a retention rule, TensorHub excludes replicas hosted on spot instances.

In the pathological case where the last non-spot replica of a retained version fails, the system can no longer serve that version. In that event, TensorHub returns a graceful error indicating that the version is unavailable, and the client can retry later on another version. This behavior is acceptable in RL training because a new version will be trained and published shortly. The consequence of this corner case is a brief delay, not loss of training progress.

Reference Server Failure. Because the reference server maintains only soft states (e.g., references), recovery is simple and lightweight. Upon detecting a server failure, a client resets its state to unpublished and connects to a preconfigured backup server. Critically, this new server does not need to recover any states from its predecessor; it simply waits to be populated by the next round of publishing from trainers. Before the new server is populated, rollouts continue using the existing weights without interruption. Eventually, all clients would notice the failure and switch to the new server.

4.6 Implementation

TensorHub is implemented with 3.7 K lines of Python and 2.4 K lines of C++ code. The reference server is implemented as a Ray [21] named actor, which is instantiated lazily upon the first client calls `open()`. The clients and the server communicate over Ray RPCs. TensorHub transfer engine is built on top of the Mooncake transfer engine [26] for RDMA.

Consistency Testing. To validate the server’s consistency guarantees, we construct unit tests in which a single test process issues requests on behalf of multiple clients to deterministically simulate arbitrary request interleavings. Failures are injected by simply halting requests for a given client. These tests operate solely at the reference-request layer, without involving data transfer, and therefore do not require RDMA NICs or GPUs. Because all requests originate from a single process, the resulting executions are deterministic and reproducible, greatly simplifying debugging. This approach to simulated concurrency testing is inspired by FoundationDB [46], which has demonstrated in production its effectiveness at uncovering subtle concurrency bugs in distributed systems.

End-to-End Checksum. To verify end-to-end correctness, we further implement checksums. Upon publishing, the client computes the checksum of each tensor and attaches it to the reference. When another client acquires this reference as a data source, it validates the checksum after transfer. Since

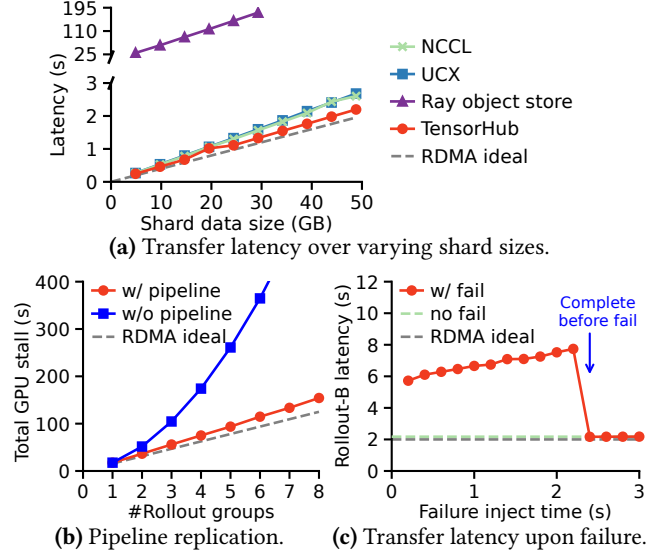


Figure 7. Microbenchmark Results.

tensors typically reside on GPUs, checksum computation is fast and can often be overlapped with RDMA transfer.

In addition, these checksums also help confirm correct enforcement of the mutability contract without corruption.

5 Evaluation

Our evaluation answers four questions: (1) *Can TensorHub transfer weight tensors efficiently and fully utilize RDMA bandwidth?* (2) *Can TensorHub scale replication throughput under bursty requests?* (3) *Can TensorHub masks failure transparently?* (4) *Does TensorHub uniformly handle a variety of realistic RL workloads?*

We begin with microbenchmarks to evaluate TensorHub’s data-plane efficiency, scalability under bursts, and resilience to failure. We then present three case studies of distinct rollout workloads, focusing on (i) standalone rollouts, (ii) elastic rollouts on spot instances, and (iii) cross-datacenter rollouts, showing how TensorHub optimizes transfer with topology, supports dynamic cluster, and minimizes worker coordination, all without extra data movement or space.

Correctness and Validation. We validate the correctness of all experiments using the mechanisms described in §4.6. We implemented 1.3K lines of concurrency tests that cover model-parallel consistency and failure cases. Each experiment runs a separate pass to verify end-to-end checksums on weights, and we trained real models to confirm loss and accuracy match those from the ByteDance production. We omit detailed results due to space.

Hardware Specification. Unless otherwise specified, our experiments use machines with NVIDIA Hopper GPUs and Mellanox InfiniBand RDMA NICs; each machine has 160 CPUs, 8 GPUs, 1800 GB Memory, 4 RDMA NICs (400 Gbps), and 1 VPC NIC (200 Gbps).

5.1 Microbenchmarks

Through microbenchmarks, we evaluate TensorHub’s data-plane efficiency, scalability under bursts, and resilience to failure. In all microbenchmarks, each worker group has 8 shards and runs on an 8-GPU machine. We have also measured the latency of control-plane operations (publish, query) and confirmed they are lightweight (at most a few milliseconds). We omit those results due to space limitations.

5.1.1 RDMA Bandwidth Efficiency. In the first microbenchmark, one machine serves as the trainer group and sends tensors, while another serves as the rollout group and receives tensors from the corresponding shards. We vary the number of tensors per shard to control shard size (50 MB per tensor) and measure the latency required to complete the transfer. We compare with NCCL [23], UCX [31], and the Ray Plasma object store [21], and show an additional *RDMA ideal* curve that denotes the theoretical number if the RDMA bandwidth is fully saturated (25 GB/s per shard).

Figure 7 shows the results, where TensorHub consistently delivers the lowest latency. The Ray Plasma object store is two orders of magnitude slower than other approaches because it is primarily optimized for CPU memory and TCP, incurring prohibitive costs for GPU-CPU movement and serialization; in addition, Ray crashes when the per-shard size exceeds 35 GB. Focusing on the three direct transfer approaches, TensorHub transfers 50 GB data in 2.2s with 22 GB/s throughput (88% of the theoretical), while NCCL and UCX only achieve 18.8 GB/s and 18.1 GB/s. These results show TensorHub’s data-plane implementation is highly efficient and can fully saturate RDMA bandwidth.

5.1.2 Scaling with Bursts. In the second microbenchmark, we evaluate TensorHub’s scalability under bursty request arrival. We started one machine for a trainer group and 1~8 machines for rollout groups; each shard contains 50 GB data. All rollouts simultaneously request replications.

We define the *total GPU stall time* as the sum of stall times over all GPUs involved in the replication. This metric reflects the aggregated computational resource loss across the cluster. We report the total GPU stall time for both the default TensorHub and the one with pipeline replication disabled, together with the RDMA ideal number for the reference.

Figure 7b shows total GPU stall time. With pipeline replication, the trainer group serves only one rollout group, and the remaining rollouts fetch peer-to-peer. Each shard transfers 50 GB in 2.2s, independent of replica count. Total stall time scales linearly with the group count and remains close to the RDMA roofline. Without pipeline replication, all replicas fetch from the trainer, causing bandwidth contention. Stall time grows quadratically with group count. This scaling gap demonstrates that pipeline replication is necessary for efficient weight dissemination at scale.

Model size	9B	36B	260B	mocked 1T
# shards	2	4	8	16
Shard size (GB)	10	19	34	66
Trainer #GPUs	16	16	64	768
Standalone #GPUs	8	8	16	256

Table 3. Training Workload Parameters.

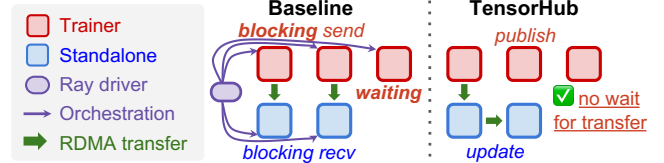


Figure 8. Weight Transfer Flows with Standalone. TensorHub does not require the Ray driver to orchestrate weight transfer. Each standalone rollout pulls weight on demand.

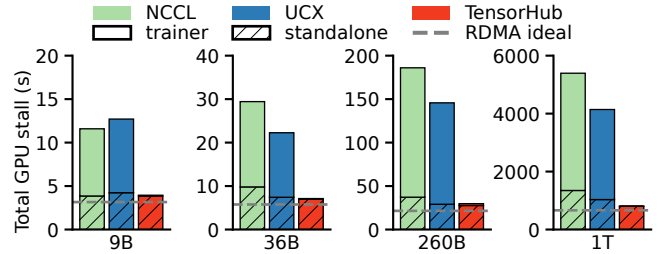


Figure 9. Standalone Rollout Results. Ideally, only standalone rollouts need to stall due to their dependence on weights; trainers can proceed without waiting. TensorHub not only eliminates trainer stalls, but also keeps standalone stall time consistently lower than alternatives.

5.1.3 Transparent Failure Masking. We evaluate the system’s resilience to failure with the third microbenchmark. Since failure at rest does not impact performance, we focus on failure during a transfer. A resilient system should mask failure without disrupting other healthy workers.

To evaluate this, we construct a pipeline where a trainer group sends data to a rollout group A, which then forwards data to another rollout group B. Both rollout groups start simultaneously. Each shard has 50 GB data. We inject a failure into rollout-A sometime after the transfer has begun, and then measure whether rollout-B can complete the transfer without interruption, as well as the total time required.

Figure 7c presents the results, where rollout-B always completes the transfer. When the failure occurs immediately after the transfer starts, rollout-B quickly detects it and retries. The RDMA layer takes a conservative timeout for ~4 seconds before B confirms A’s failure and requests recovery from the server. The server schedules B to directly read from the trainer to complete the transfer. For failures injected at later points, B takes longer to complete because retransmission is delayed. If the failure is introduced after 2.2s, rollout-B has already finished the transfer and is therefore unaffected. These results demonstrate that TensorHub can transparently mask failure with only minor delay.

5.2 RL Training with Standalone Rollouts

We now evaluate TensorHub in realistic RL settings. We use ByteDance production framework built on top of veRL [34]. veRL relies on a Ray driver to orchestrate workers, which is a common and representative design of RL frameworks [7, 12]. **Baseline: NCCL and UCX.** We compare against two weight transfer systems used in ByteDance production stack: one based on NCCL [23] and another based on UCX [31]. As discussed in §2.3, the communication-based weight transfer systems require the RL framework to coordinate workers. In practice, it is implemented as an RPC broadcast that interrupts all workers for a global weight transfer stage. More fine-grained coordination is theoretically possible, but it requires a major refactor of the framework. Though we are unable to compare with every possible implementation, we again include a roofline curve for reference (labeled “RDMA ideal”), denoting an ideal case with zero coordination cost or contention, and the only bottleneck is the rollouts’ RDMA bandwidth. Performance close to this reference implies near-optimal results. We did not include a comparison with other storage-based approaches because we have shown they are an order of magnitude slower in §5.1.

Engineering Effort for Integration. NCCL requires ~450 lines of code changes to ByteDance production RL framework, mostly for coordination. UCX, with more low-level control, requires ~1200 lines, trading simplicity for flexibility. TensorHub requires only ~40 lines of change (similar to Figure 4 example). This highlights that TensorHub offers a simple and user-friendly interface.

Workloads. We trained four models for evaluation (Table 3), covering various sharding and GPU count setups. The 9B, 36B, and 260B models are initialized from pretrain weights. To stress the system at an extreme scale, we construct a 1T-parameter model with mocked weight data by duplicating the 260B-parameter model’s layer weights four times: training such a huge model requires 768 GPUs for trainers and 256 GPUs for standalone rollouts (1024 GPUs in total).

Results. Figure 8 illustrates the workflow, and Figure 9 reports total GPU stall time per step. TensorHub significantly reduces stall time due to its minimal coordination design. First, publish is a lightweight reference-passing request that does not wait for actual data transfer; trainers immediately resume for their own co-located rollout tasks. In contrast, NCCL and UCX require framework Ray driver coordination, and the driver does not move forward to its next stage until the weight transfer stage is completed.

Second, for large models, stragglers are amplified at scale: the weight-transfer stage completes only when the slowest worker finishes. For the 1T model, transferring 66 GB should ideally take 2.6s at full bandwidth, but NCCL and UCX stall all 1024 GPUs for 5.3s and 4.0s, respectively. TensorHub localizes the stall: mean latency is 3.1s, and only the standalones

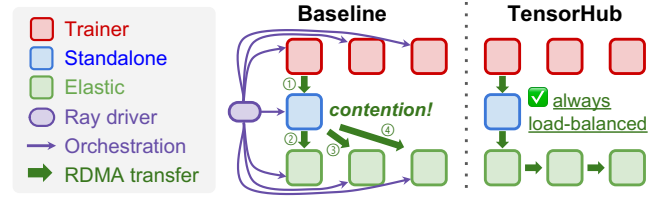


Figure 10. Weight Transfer Flows with Standalone and Elastic. Note TensorHub only shows one possible data flow; an elastic can also fetch weights from a trainer.

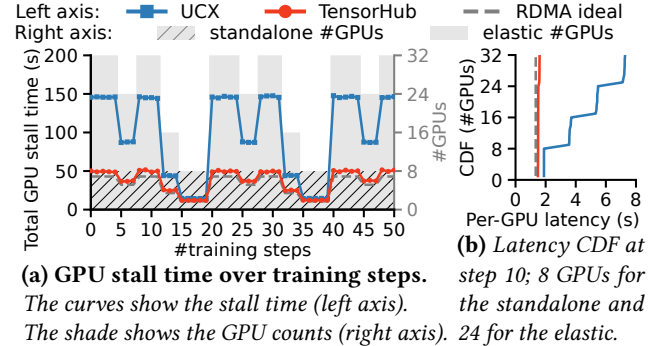


Figure 11. Elastic Rollout Results.

need to wait. This results in up to $6.7\times$ total GPU stall time reduction compared to NCCL.

5.3 Elastic Rollout on Spot Instances

In the second case study, we evaluate TensorHub when rollout workers run on a hybrid of stable GPUs (for standalone) and preemptible spot GPUs (for elastic). In ByteDance production environment, an autoscaler monitors training progress and instantiates new elastic rollouts when rollout tasks are backlogged and spare spot GPUs are available. When their GPU resources are preempted, elastic rollouts are killed immediately without a grace period.

Baseline. Only UCX-based weight transfer supports elastic rollout, because NCCL assumes static membership. Supporting elastic rollouts took substantial engineering effort for ByteDance production UCX-based implementation. When a new elastic worker appears, trainers may be mid-training and unable to serve weights, so standalone rollouts must also act as data senders. This introduces a trainer to standalone to elastic transfer topology.

Workloads. We use the 260B model described in Table 3, with one standalone machine (8 GPUs) and three elastic machines (24 GPUs). To keep experiments reproducible, we intercept the autoscaler and deterministically issue scale-up and scale-down events. Random elastic replicas are terminated during scale-down.

Results. Figure 10 illustrates the workflow, and Figure 11a shows total stall time over training steps with GPU count varied. Since trainers do not stall in TensorHub (§5.2), we focus on stall time across standalone and elastic GPUs.

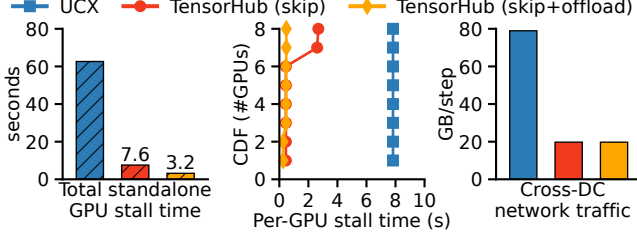


Figure 12. Cross-Datcenter Rollout Results. Note that the left figure shows only the standalone stall time; UCX incurs additional trainer stall time that is omitted here.

In the UCX baseline, elastic rollouts must wait for standalone rollouts to pull weights from trainers first, introducing a delay. When elastic workers outnumber standalones, bandwidth contention further amplifies tail latency. Figure 11b shows per-GPU stall time at step 10: UCX produces a stair-shaped CDF, with the last batch of elastic GPUs waiting up to 7.2s for weights.

With TensorHub, the reference server balances topology at runtime, so elastic rollouts can fetch weights directly from trainers, standalones, or peer elastic rollouts, and pipeline replication spreads load across all available replicas. As a result, stall time remains near-constant (~ 1.5 s) independent of the elastic GPU count, approaching the RDMA ideal. This result demonstrates that TensorHub naturally accommodates dynamic membership and runtime topology.

5.4 Cross-Datcenter RL Training

The final case study evaluates TensorHub in a multi-datcenter deployment. This setting reflects production scenarios where spare inference resources are available in a separate datcenter, and utilizing them requires weight transfers over slower inter-datcenter networks.

Baseline. We use a UCX-based implementation with TCP transport as the baseline. The transfer topology is similar to standalone rollouts (Figure 8), except the data path is now cross-datcenter over TCP.

Workloads. We use the 9B model from Table 3, with 16 GPUs for trainers and 8 GPUs for standalone rollouts. Trainers are located in a datcenter with training-optimized Hopper GPUs; standalone rollouts run in a second datcenter with inference-optimized Hopper GPUs. The two datcenters are only reachable via the 200 Gbps VPC NIC (one per machine) with TCP.

Results. Figure 12 reports total GPU stall time, per-GPU latency distribution, and total cross-datcenter TCP traffic. With the UCX baseline, every rollout replica fetches weights directly over TCP. Since all flows contend for the single VPC NIC per machine, stall time is dominated by cross-datcenter bandwidth: each GPU waits 7.8 seconds for weights.

With TensorHub and smart skipping, only the first standalone replica (2 GPUs) waits for cross-datcenter TCP transfer (2.5s), and the contention on the VPC NIC is reduced.

Other standalone replicas can fetch data via high-throughput RDMA with merely 0.45s latency. The result is a latency distribution with a single long tail for the seeding replica and near-ideal RDMA latency for all others.

With offload seeding enabled, TensorHub further hides the first transfer by using an offload replica: the client creates an offload buffer in CPU memory and returns to computation, allowing the data transfer to proceed in the background. This trades a bounded CPU memory overhead (one copy per datcenter) for eliminating foreground stall. In summary, TensorHub achieves cross-datcenter weight transfer with near-local performance after a single seeding transfer.

6 Related Work

Reinforcement Learning for LLM. Many frameworks [12, 34, 45] have been proposed to accelerate RL training. verL [34] combines single-controller and multi-controller paradigms to drive stage execution efficiently. RLHFuse [45] further improves training compute efficiency with stage fusion. RollPacker [8] optimizes the long-tail decoding in rollout, while SpecActor [3] introduces decoupled and Best-of-N speculation to accelerate the rollout phase, ensuring on-policy model training. RLBoost [40] harvests preemptible GPUs to speed up the rollout phase with resource elasticity. Recent work also explores off-policy approaches for training acceleration. AsyncRLHF [22], AReaL [7], StreamRL [44], and Laminar [33] introduce asynchronous RL post-training with tailored optimizations to increase job throughput. TLT [13] trains a draft model in parallel for speculation when RL model training is performed; thus, the draft model also requires weight synchronization.

Reinforcement learning systems are progressively evolving towards architectures with complex control flow interactions among multiple modules, aiming to either enhance model performance or achieve acceleration. TensorHub has identified the challenges posed by the rapid escalation of such complexity to the scalability and stability of distributed systems. Drawing inspiration from the design principles of classical distributed filesystems [2, 9] and peer-to-peer storage [35], it introduces an abstraction layer with a storage interface. Through streamlined APIs, it can support complex scenarios such as asynchronous training, elastic rollout, and even heterogeneous GPU types across datcenters, while achieving excellent performance.

Storage Systems for Machine Learning. Prior studies have explored techniques for building storage systems for machine learning workloads. Parameter Server [17] is a classic system architecture for distributed machine learning, which employs a cluster of server nodes to store the model states. It does not extend its success to LLM training due to costly data movement. Ray [21] provides Plasma object store for RL workloads, but is primarily optimized for CPU workloads. GeminiFS [27] accelerates checkpointing and

activation offloading by optimizing the metadata through a POSIX-like interface. GoFS [18] further offloads the entire storage management functionality to the GPU, enabling GPU programs to access storage with direct scalability. As for LLM pretraining, ByteCheckpoint [37] is designed for efficient model weight storage and supports dynamic resharding across different training parallelism. TensorHub is tailored for dynamic weight transfer in LLM RL training. By introducing a concise storage abstraction and a reference-oriented system architecture, it achieves better adaptability to fault tolerance, elasticity, and cross-datacenter deployment.

7 Conclusion

We present Reference-Oriented Storage (ROS), a new approach to weight transfer in large-scale LLM reinforcement learning systems. By eliminating explicit data ownership and instead leveraging the inherent redundancy and immutability of model weights, ROS reconciles a long-standing trade-off between flexibility and efficiency in RL weight-transfer systems. Building on this abstraction, TensorHub enables topology-aware, low-overhead weight transfer with minimal coordination, while preserving consistency and fault tolerance in dynamic environments. Our evaluation shows that TensorHub can fully saturate network bandwidth and deliver near-optimal end-to-end performance. TensorHub has been deployed in ByteDance production for cutting-edge RL training.

References

- [1] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. 2022. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. *arXiv:2204.05862* [cs.CL] <https://arxiv.org/abs/2204.05862>
- [2] Dhruva Borthakur. [n. d.]. HDFS Architecture Guide. https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html
- [3] Rongxin Cheng, Kai Zhou, Xingda Wei, Siyuan Liu, Mingcong Han, Mingjing Ai, Yeju Zhou, Baoquan Zhong, Wencong Xiao, Rong Chen, and Haibo Chen. 2025. Fast LLM Post-training via Decoupled and Best-of-N Speculation. *arXiv:2511.16193* [cs.DC] <https://arxiv.org/abs/2511.16193>
- [4] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (*NIPS'17*). Curran Associates Inc., Red Hook, NY, USA, 4302–4310.
- [5] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [6] Josef Dai, Xuehai Pan, Ruiyang Sun, Jiaming Ji, Xinbo Xu, Mickel Liu, Yizhou Wang, and Yaodong Yang. 2023. Safe RLHF: Safe Reinforcement Learning from Human Feedback. *arXiv:2310.12773* [cs.AI] <https://arxiv.org/abs/2310.12773>
- [7] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. 2025. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. *arXiv:2505.24298* [cs.LG] <https://arxiv.org/abs/2505.24298>
- [8] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, Jiamang Wang, Lin Qu, Bo Zheng, and Wei Wang. 2025. RollPacker: Mitigating Long-Tail Rollouts for Fast, Synchronous RL Post-Training. *arXiv:2509.21009* [cs.DC] <https://arxiv.org/abs/2509.21009>
- [9] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. 2003. The Google file system. In *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles* (Bolton Landing, NY, USA) (*SOSP '03*). Association for Computing Machinery, New York, NY, USA, 29–43. doi:10.1145/945445.945450
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. 2025. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature* 645, 8081 (2025), 633–638.
- [11] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. 2025. History Rhymes: Accelerating LLM Reinforcement Learning with RhymeRL. *arXiv:2508.18588* [cs.LG] <https://arxiv.org/abs/2508.18588>
- [12] Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, Weikai Fang, Xianyu, Yu Cao, Haotian Xu, and Yiming Liu. 2025. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. *arXiv:2405.11143* [cs.AI] <https://arxiv.org/abs/2405.11143>
- [13] Qinghao Hu, Shang Yang, Junxian Guo, Xiaozhe Yao, Yujun Lin, Yuxian Gu, Han Cai, Chuang Gan, Ana Klimovic, and Song Han. 2025. Taming the Long-Tail: Efficient Reasoning RL Training with Adaptive Drafter. *arXiv:2511.16665* [cs.LG] <https://arxiv.org/abs/2511.16665>
- [14] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [15] Yimin Jiang, Yibo Zhu, Chang Lan, Bairen Yi, Yong Cui, and Chuanxiong Guo. 2020. A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 463–479. <https://www.usenix.org/conference/osdi20/presentation/jiang>
- [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) (*SOSP '23*). Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [17] Mu Li, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Yiing Su. 2014. Scaling Distributed Machine Learning with the Parameter Server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 583–598. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/li_mu
- [18] Shaobo Li, Yirui Eric Zhou, Yuqi Xue, Yuan Xu, and Jian Huang. 2025. Managing Scalable Direct Storage Accesses for GPUs with GoFS. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) (*SOSP '25*). Association for Computing Machinery, New York, NY, USA, 979–995. doi:10.1145/3731569.3764857

- [19] Shaoteng Liu, Haoqi Yuan, Minda Hu, Yanwei Li, Yukang Chen, Shu Liu, Zongqing Lu, and Jiaya Jia. 2024. RL-gpt: Integrating reinforcement learning and code-as-policy. *Advances in Neural Information Processing Systems* 37 (2024), 28430–28459.
- [20] Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. 2025. RealL: Efficient RLHF Training of Large Language Models with Parameter Reallocation. arXiv:2406.14088 [cs.DC] <https://arxiv.org/abs/2406.14088>
- [21] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 561–577. <https://www.usenix.org/conference/osdi18/presentation/moritz>
- [22] Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hoseini, Rishabh Agarwal, and Aaron Courville. 2025. Asynchronous RLHF: Faster and More Efficient Off-Policy RL for Language Models. arXiv:2410.18252 [cs.LG] <https://arxiv.org/abs/2410.18252>
- [23] Nvidia. [n. d.]. Nvidia/NCCL: Optimized Primitives for collective multi-gpu communication. <https://github.com/NVIDIA/nccl>
- [24] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 27730–27744. https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf
- [25] Dylan Patel, Daniel Nishball, and Jeremie Eliahou Ontiveros. 2024. Multi-Datacenter Training: OpenAI’s Ambitious Plan to Beat Google’s Infrastructure. <https://newsletter.semianalysis.com/p/multi-datacenter-training-openais>
- [26] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170. <https://www.usenix.org/conference/fast25/presentation/qin>
- [27] Shi Qiu, Weinan Liu, Yifan Hu, Jianqin Yan, Zhirong Shen, Xin Yao, Renhai Chen, Gong Zhang, and Yiming Zhang. 2025. GeminiFS: A Companion File System for GPUs. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 221–236. <https://www.usenix.org/conference/fast25/presentation/qiu>
- [28] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [29] Amrith Setlur, Saurabh Garg, Xinyang Geng, Naman Garg, Virginia Smith, and Aviral Kumar. 2024. RL on incorrect synthetic data scales the efficiency of llm math reasoning by eight-fold. *Advances in Neural Information Processing Systems* 37 (2024), 43000–43031.
- [30] Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. 2024. Rewarding progress: Scaling automated process verifiers for llm reasoning. *arXiv preprint arXiv:2410.08146* (2024).
- [31] Pavel Shamis, Manjunath Gorentla Venkata, M Graham Lopez, Matthew B Baker, Oscar Hernandez, Yossi Itigin, Mike Dubman, Gilad Shainer, Richard L Graham, Liran Liss, et al. 2015. UCX: an open source framework for HPC network APIs and beyond. In *2015 IEEE 23rd Annual Symposium on High-Performance Interconnects*. IEEE, 40–43.
- [32] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] <https://arxiv.org/abs/2402.03300>
- [33] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. 2025. Laminar: A Scalable Asynchronous RL Post-Training Framework. arXiv:2510.12633 [cs.LG] <https://arxiv.org/abs/2510.12633>
- [34] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2025. HybridFlow: A Flexible and Efficient RLHF Framework. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) (*EuroSys ’25*). Association for Computing Machinery, New York, NY, USA, 1279–1297. doi:10.1145/3689031.3696075
- [35] Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan. 2001. Chord: A scalable peer-to-peer lookup service for internet applications. *SIGCOMM Comput. Commun. Rev.* 31, 4 (Aug. 2001), 149–160. doi:10.1145/964723.383071
- [36] Grok 4 Team. 2025. Grok 4 Report. *xAI grok4 news* (2025). <https://x.ai/news/grok-4>
- [37] Borui Wan, Mingji Han, Yiyao Sheng, Yanghua Peng, Haibin Lin, Mofan Zhang, Zhichao Lai, Menghan Yu, Junda Zhang, Zuquan Song, et al. 2025. {ByteCheckpoint}: A Unified Checkpointing System for Large Foundation Model Development. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 559–578.
- [38] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I Wang. 2025. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449* (2025).
- [39] Bo Wu, Sid Wang, Yunhao Tang, Jia Ding, Eryk Helenowski, Liang Tan, Tengyu Xu, Tushar Gowda, Zhengxing Chen, Chen Zhu, Xiaocheng Tang, Yundi Qian, Beibei Zhu, and Rui Hou. 2025. LlamaRL: A Distributed Asynchronous Reinforcement Learning Framework for Efficient Large-scale LLM Training. arXiv:2505.24034 [cs.LG] <https://arxiv.org/abs/2505.24034>
- [40] Yongji Wu, Xueshen Liu, Haizhong Zheng, Juncheng Gu, Beidi Chen, Z. Morley Mao, Arvind Krishnamurthy, and Ion Stoica. 2025. RLBoost: Harvesting Preemptible Resources for Cost-Efficient Reinforcement Learning on LLMs. arXiv:2510.19225 [cs.DC] <https://arxiv.org/abs/2510.19225>
- [41] Jianhao Yan, Yafu Li, Zican Hu, Zhi Wang, Ganqu Cui, Xiaoye Qu, Yu Cheng, and Yue Zhang. 2025. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945* (2025).
- [42] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476 [cs.LG] <https://arxiv.org/abs/2503.14476>
- [43] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
- [44] Yinmin Zhong, Zili Zhang, Xiaoni Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, Hongyu Zhou, Yimin Jiang, Yibo Zhu, and Daxin Jiang. 2025. StreamRL: Scalable, Heterogeneous, and Elastic RL for LLMs with Disaggregated

Stream Generation. arXiv:2504.15930 [cs.LG] <https://arxiv.org/abs/2504.15930>

- [45] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, and Xin Jin. 2025. Optimizing RLHF Training for Large Language Models with Stage Fusion. In *22nd USENIX Symposium on Networked Systems Design and Implementation, NSDI 2025, Philadelphia, PA, USA, April 28-30, 2025*, Theophilus A. Benson and Radhika Niranjan Mysore (Eds.). USENIX Association, 489–503. <https://www.usenix.org/conference/nsdi25/presentation/zhong>
- [46] Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton, Andrew J. Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Xiaoge Su, and Vishesh Yadav. 2021. FoundationDB: A Distributed Unbundled Transactional Key Value Store. In *Proceedings of the 2021 International Conference on Management of Data (Virtual Event, China) (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2653–2666. doi:10.1145/3448016.3457559
- [47] Yuzhen Zhou, Jiajun Li, Yusheng Su, Gowtham Ramesh, Zilin Zhu, Xiang Long, Chenyang Zhao, Jin Pan, Xiaodong Yu, Ze Wang, et al. 2025. April: Active partial rollouts in reinforcement learning to tame long-tail generation. *arXiv preprint arXiv:2509.18521* (2025).
- [48] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. 2025. slime: An LLM post-training framework for RL Scaling. <https://github.com/THUDM/slime>. GitHub repository. Corresponding author: Xin Lv.